Nomad: an inspectable research-mapping pipeline

Multi-query retrieval, source-linked ranking and the limits of deterministic test evidence

Mergen Ulziibayar
New York University, Mechanical Engineering
13 November 2026

Software engineering report. Source snapshot: `be6bf7c836e1`, 11 November 2026. Retained test run: 5 August 2026. Working application; relevance and user benefit remain unevaluated.

Abstract

Nomad is a Next.js and TypeScript application that converts an engineering query into a source-linked starting map of papers, researchers, topic clusters, project directions and a reading roadmap. It retrieves up to three OpenAlex query variants, normalizes and deduplicates records, computes explicit ranking signals, and derives classic and explorer views from a common response. The deterministic path operates without a paid model key; embeddings and an optional relevance judge augment relevance but do not create bibliographic source records. A retained run reports 88 passing tests in nine files. Inspection of the test protocol shows that only two of five named topic fixtures exist and three topic tests return before their assertions, while still appearing as passes. Accordingly, the results support deterministic component behavior and two narrow topic invariants, not precision, recall, ranking superiority or research productivity. This report documents implemented equations, request and data boundaries, view semantics, failure handling, evidence provenance and a prospective evaluation capable of answering the remaining quality question. The central result is an implemented and inspectable pipeline, with a deliberately narrower claim than an independently validated literature recommender.

Completed scope. This report closes the implemented retrieval-and-mapping software project at commit `be6bf7c836e1`. The frozen scope includes its deterministic transformations, user-facing views and the retained test record. Relevance, ranking superiority, optional model-path effectiveness and researcher benefit remain unmeasured. The three missing topic captures are recorded as inactive assertion coverage rather than silently counted as substantive topic validations.

1 Question and engineering objective

Central question. Can a broad engineering literature query be turned into a structured, traceable starting map without hiding where the records and ordering come from? **Recorded answer.** The repository implements that transformation and tests many of its deterministic components. It does not yet answer whether the resulting selections are more relevant, complete or useful than a simple search baseline.

The design hypothesis is that coverage and inspectability should be handled separately: first retrieve a broader candidate set with explicit query variants, then rank and organize it using disclosed signals and preserve source links. The application should degrade to a deterministic metadata/keyword path when optional semantic services are unavailable. Its outputs are reading-order and exploration aids, not claims that citation popularity measures scientific truth or predicts future impact.

The portfolio identifies the owner's contribution as full-stack design, query-variant retrieval, OpenAlex normalization/deduplication/filtering, ranking and clustering, the classic map and v2 explorer, and the deterministic fixture/test suite. The implementation was developed with AI assistance in the broader portfolio workflow. The source does not establish a quantitative division of human and AI-authored code; no such percentage is asserted. Codex prepared this report from code and retained documents. No live retrieval campaign, fresh test run, user study or measured relevance evaluation was performed

to create it.

1.1 Context and source model

OpenAlex provides a bibliographic graph of works, authors, institutions, sources and topics [2]. Its work records include identifiers, dates, abstracts when available, citations and reference relationships [3]. That upstream graph is the source of bibliographic facts in Nomad. The application adds query expansion, ranking heuristics and derived interfaces; it is not a new scholarly index. OpenAlex metadata, citation counts and search order can change, so a reproducible code revision alone cannot reproduce an old live result.

The report treats the application as software engineering with an evaluation gap, rather than a demonstrated information-retrieval advance. It contains no comparison with Google Scholar, Connected Papers, OpenAlex search or another mapping tool. Such a comparison would require shared requests, independently judged relevance labels and a controlled protocol, rather than selected screenshots.

2 Architecture and contracts

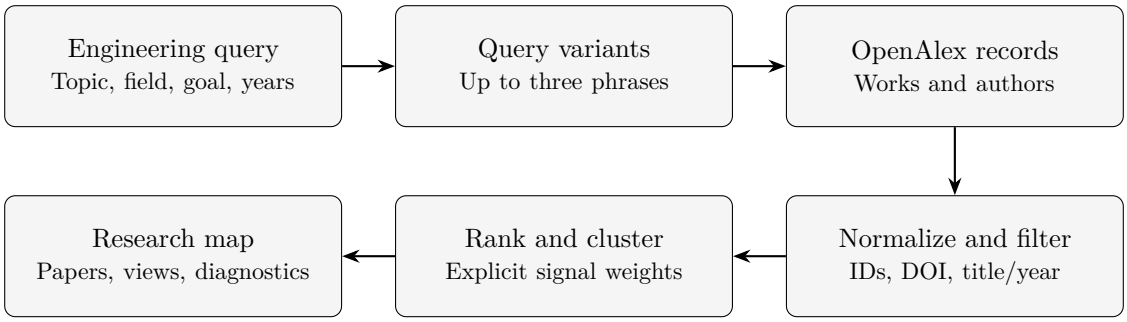


Figure 1: Source-derived architecture, redrawn for legibility from the portfolio pipeline diagram. This is a pipeline explanation, not a product screenshot, measured result or reconstruction of the full citation network. OpenAlex supplies records; ranking and map views are downstream transformations.

The stack uses Next.js 15, React 19, TypeScript, Zod for schema work and Vitest for tests, as recorded in the package manifest and lockfile. The classic interface is served at the root; the explorer offers graph, mind-map, roadmap, trend and quality views. API routes produce a typed research-map response. The principal code boundaries are shown in Table 1.

Table 1: Implemented component boundaries at the pinned source snapshot. Paths are repository-relative.

Component	Responsibility
`lib/openalex.ts`	Query URLs/variants, upstream fetching, work-ID merge, cache and citation history
`lib/scoring.ts`	Record normalization, deduplication, explicit ranking, clusters and project synthesis
`lib/embeddings.ts`	Optional semantic relevance and deterministic fallback
`lib/relevance-judge.ts`	Optional separately gated relevance-grading path
`lib/quality/`	Drift diagnostics, subclusters, regression/evaluation and v2 quality report
`lib/derive/`	Graph, mind map, roadmap, trends and explanatory structures
`app/explore/`	Alternate visual views of a common map response
`test-fixtures/openalex/`	Frozen responses for repeatable regression checks

The request includes topic, optional field, research goal and year interval. Search URLs constrain publication dates and exclude retracted records. Selected fields cover work IDs and DOI, title, publication

date/year, citation counts, normalized citation percentile, authorships, topics, keywords, work type, retraction status, abstract inverted index, references, yearly counts and related metadata. The default request asks for 200 works per query variant. A URL helper clamps its per-page argument to 30–300, which is a code behavior and not evidence that every upstream setting has been validated against current service rules.

The response carries more than ranked paper names. It includes source-linked recommendations, clusters, people, project directions, data-quality diagnostics and explanations. All visualizations derive from that response, so agreement between graph and roadmap is internal consistency, not an independent validation of the papers selected.

3 Retrieval and normalization methods

3.1 Coverage before ranking

The retrieval layer builds an ordered list capped at three search strings: the user's exact topic, a field-scoped phrase when a field exists, and a core-term phrase with filler removed. Empty or case-insensitive duplicate strings are excluded. Keeping the verbatim query first preserves its ordering preference when results are merged. Multi-query retrieval is intended to broaden candidate coverage; no recall improvement has been measured against a gold standard.

The first deduplication layer normalizes OpenAlex work IDs and keeps the first occurrence. Records missing IDs are retained at this stage. The normalization layer then rejects retracted works, missing titles and records outside the requested year range. It creates a second deduplication key from normalized DOI when available, otherwise normalized title and year. This catches some duplicated publications beyond ID equality, but the fallback can also merge distinct works with identical normalized titles/years. No empirical false-merge or missed-duplicate rate is available.

Abstract text is reconstructed from OpenAlex's inverted index. Missing abstract availability is recorded, rather than silently represented as evidence that the title captures the whole paper. Metadata defaults such as citation count zero and missing publication year need care: a zero can mean absence of information rather than a true bibliometric observation. Filtering, exclusion counts and data-quality fields make some of these conditions inspectable, but they do not repair the upstream record.

3.2 Caching, request bounds and failure semantics

Work and citation-history caches are in-memory maps with a one-hour time-to-live. They reduce repeated calls in the same process but are neither persistent provenance stores nor guarantees of identical responses across deployments. Network fetches use no-store at the fetch layer; a failed work/reference HTTP response raises an error including its status. Citation-history paths have explicit fallbacks and reporting, so a missing history need not be fabricated as a complete time series.

The ranking code declares a seven-request budget for its enrichment stage, a shared-reference minimum frequency of two, and a fetch cap of twenty shared references. These constants constrain work, but are not a measured end-to-end latency or reliability service level. No response-time distribution, peak memory benchmark, availability trial or concurrency stress test is retained. The application depends on upstream connectivity and metadata service behavior; a complete deterministic fixture test is intentionally different from exercising that dependency live.

4 Inspectable ranking equations

The principal ranking formulas are encoded directly in `scoreWorks`. Let L denote the candidate-set percentile of $\log(1 + c)$, P an available normalized citation percentile (otherwise a local citation-count

percentile), R final relevance, and S a source-quality heuristic. The foundational score is

$$F = 0.40L + 0.30P + 0.20R + 0.10S. \tag{1}$$

Shared-reference graph support adds G, capped by the code at 0.05, to produce the final foundational rank $F + G$. For citations-per-year percentile C, recency A and the same relevance/percentile terms, the recent-influence score is

$$I = 0.30C + 0.25A + 0.30R + 0.15P. \tag{2}$$

These weights are implemented heuristics, not fitted effect estimates or validated importance coefficients. Scores are relative to the retrieved set. Adding candidates can change local percentiles and thus ordering even when a work's own metadata has not changed.

Recency uses the requested year window while citations per year uses the runtime calendar year:

$$A = 1 - \mathrm{clip}_{[0,1]}\left(\frac{y_{to} - y}{\max(1, y_{to} - y_{from})}\right), \qquad c_{yr} = \frac{c}{\max(1, y_{now} - y + 1)}. \tag{3}$$

Consequently, frozen records plus a code commit do not fully freeze scores unless the runtime date is also controlled. The age normalization also favors different papers in different requested windows. These are reproducibility inputs, not incidental presentation details.

4.1 Optional semantic and judge signals

Without a semantic key, keyword relevance K remains the relevance signal. If an embedding is available, the code uses a normalized cosine similarity $M = (1 + \cos(v_q, v_p))/2$ and combines it as

$$R_0 = 0.65K + 0.35M. \tag{4}$$

When a judge grade J is available, the alternative combination is

$$R_0 = 0.55J + 0.30K + 0.15\widetilde{M}, \tag{5}$$

where $\widetilde{M} = M$ when present and K otherwise. A detected pre-gate drift domain multiplies relevance by 0.85, followed by clipping to $[0, 1]$. The distinction between soft demotion and optional hard filtering matters: a diagnostic label does not necessarily remove a paper.

The inspected embedding module selects at most 200 works, truncates input text to 2,800 characters and caches vectors by work/model/text hash. The recorded model identifier is `text-embedding-3-small`. Its presence describes implemented code, not a current model recommendation or a measured quality result. Missing key, absent vectors and optional failures leave deterministic fallback available. No retained evaluation establishes that Eq. (4), the judge path or their weight choices improve relevance.

The UI exposes ranked signal contributions and reason codes. For example, foundational contribution fields correspond to citation mass, citation percentile, topic aboutness, source heuristic and shared-reference support. The top contributions explain the arithmetic order, not why an expert would necessarily consider the paper important. A high citation-derived score can reflect field age, record coverage or a popular review instead of suitability for the user's task.

5 Clusters, research directions and visual semantics

Clusters begin from topic-label assignments and aggregate candidate statistics; later quality code can split broad clusters. They are not expert-reviewed taxonomies of a field. The knowledge graph creates topic, cluster, foundational-paper, rising-paper and author nodes, connected by typed relations. The rendered graph uses fixed radial rings with force simulation disabled after positions are assigned. Paper-to-cluster

and author-to-paper edges describe the application's derived map. They are not a reconstruction of every citation or collaboration in OpenAlex.

Shared-reference signals used in ranking are a separate calculation from those display edges. Conflating visual proximity with a quantitative bibliographic similarity would therefore be incorrect. Likewise, showing a paper in several views does not increase the evidence supporting its relevance.

The roadmap filters recommendations with a relevance floor of 0.4, prioritizes suitable review papers and foundational selections, avoids reusing IDs across phases, then picks cluster-specific readings and up to two project ideas. Phase names such as "Day 1" and "Week 2" are presentation guidance, not experimentally calibrated estimates of reading time or project feasibility.

The trend module combines recent-paper share, average recent-influence score, rising-paper contribution and relevance at weights 0.40, 0.25, 0.20 and 0.15. Default trend derivation excludes clusters with fewer than three papers. Its "growing" labels summarize a retrieved sample using a heuristic; they are not a census of the field, a causal growth model or an out-of-sample forecast. Sparse retrieval and uneven citation coverage can change those labels.

Project directions are assembled with source constraints and explicit first-experiment language. The code caps project count at five and title length at fourteen words, and checks outcomes against evidence terms to avoid unsupported application leaps. These are valuable output constraints, but a syntactically concrete proposal can still be scientifically unoriginal, infeasible or poorly matched to the source. No expert novelty or feasibility rating is retained.

6 Retained test experiment and actual coverage

6.1 Protocol and result

The committed evaluation note records `npm test -- --reporter=verbose` on 5 August 2026: nine test files, 88 tests reported passed, zero reported failures. This report cites that retained result and inspects its protocol; it does not call it a fresh run at the November source snapshot. Frozen OpenAlex captures allow deterministic checks without network changes or model keys. Their purpose is regression, not representative sampling of engineering literature.

Table 2: Retained automated evidence and its interpretation. Source: evaluation notes and test implementation.

Evidence	Recorded outcome	Supported interpretation
Test runner	88 reported passes; nine files	Includes three cases with no topic assertions exercised
Named topic captures	Two of five present	Limited topic-fixture coverage
Three absent-topic tests	Early return, shown as pass	Their named assertions were not exercised
Remaining reported cases	$88 - 3 = 85$	Case-result arithmetic, not a measured assertion count
Live retrieval quality	No judged result set	No precision, recall or quality estimate
Optional model paths	No quality campaign	No supported semantic/judge improvement claim
Researcher use	No user study	No measured time saving or selection benefit

6.2 The missing-fixture mechanism

The topic evaluation file enables its tests if any JSON fixture exists. Each named test then loads its own fixture and returns immediately if that file is absent. Vitest reports such a returned test as passed, although no topic assertion ran. This is documented openly in the repository. The nominal count therefore cannot be described as 88 substantive relevance validations. Subtracting the three known early-return cases leaves 85 other reported case results. That arithmetic is not an instrumented count of executed assertions: unit tests contain different numbers and kinds of assertions, and a pass count is not a common unit of retrieval quality.

Table 3: All five named topic audits and actual fixture status. Source: `lib/quality/eval.test.ts` and committed fixture directory.

Topic	Fixture	Active or intended invariant
Battery thermal management	Present	Dominant cluster creates at least two subclusters
Soft robotics	Present	Foundational selections retain at least one named canonical paper family
AI in mechanical design	Missing	Hospitality/marketing drift assertion not exercised
Additive-manufacturing defects	Missing	On-topic foundational-pick ratio not exercised
6G channel modeling	Missing	Channel/RIS/terahertz ratio not exercised

The cross-topic sanity check processes the two available fixtures. Passing a cluster-splitting count does not establish coherent cluster meaning; retaining one canonical family does not establish that the other selected papers are relevant. Those are intentionally narrow invariants. No paper-by-paper independent labels, relevance confusion matrix or ranked effectiveness metric is available.

6.3 What the wider suite checks

The nine-suite result covers query variants, normalization, deduplication, relevance diagnostics, score construction, project constraints, citation-history handling, shared-reference support, query focus, subclustering, roadmaps, trend calculations and response construction. These checks can detect accidental algorithm changes or malformed outputs. Their validity depends on the test oracle: testing that encoded weights produce an expected sum checks implementation consistency, while establishing that the weights serve users requires different evidence.

The suite also does not establish visual quality. A valid graph data structure may render with overlapping labels or inaccessible contrast; a sensible roadmap object may overpromise a novice's time budget. Neither usability across graph sizes nor mobile accessibility has been measured in the retained evaluation. This report does not use an architecture figure as evidence that those interfaces were tested successfully.

7 Uncertainty, failures and limitations

Uncertainty enters in at least four distinct layers. Upstream records may be missing, merged, retracted or updated; retrieval may under-cover the field; heuristic ranking can prefer popular off-topic works; and a readable map may still fail a user's research task. Unit tests address parts of transformation correctness but do not collapse the other layers into one confidence score.

The most concrete evaluation failure is incomplete activation of topic tests. It should be made opera-

tionally visible in future work with an explicit skip/missing-fixture result or a strict required-fixture gate. That recommendation is not presented as an implemented fix: changing code to make a report look more complete would obscure the actual retained evidence.

Live-result drift is expected because citation totals, records, search ordering, runtime date and model availability can change. In-memory cache identity is not a durable dataset identity. Optional semantic processing is limited to a selected candidate subset; late candidates may have different signals. Local percentile normalization makes scores candidate-dependent. All of these are material to reproduction and interpretation.

External validity is especially limited: two hand-trimmed topic captures cannot represent the breadth of mechanical engineering, multilingual literature or a newcomer's question quality. There is no measured retrieval superiority, relevance calibration, citation accuracy audit, novelty score or productivity gain. "Complete" in the portfolio describes the implemented software project, not completion of those research evaluations.

8 Reproducibility and prospective quality experiment

8.1 Recorded reproduction boundary

The inspected commit is `be6bf7c836e1cb491099bc979c0d39f731617071`. Unlike the two other software reports in this set, the portfolio did not supply an older pinned source commit for Nomad. The package lock and source snapshot establish a software basis; the test result remains the separately dated retained run. The accompanying manifest hashes the code, evaluation note and both captures, plus the copied architecture image. Source code carries an MIT license; the project-created diagram is credited to this portfolio. No third-party paper figures are reproduced.

The documented local workflow is dependency installation from the lockfile, the Next development server, and the deterministic test command. Reproducing a live view additionally requires the exact request, runtime date, environment flags, cached/upstream payloads and optional model outputs. A publication-quality comparison should retain raw response JSON with capture time and a normalized request, rather than relying on a screenshot or a mutable API URL.

8.2 A discriminating next study, not a claimed result

A prospective quality experiment should first capture all five named topics and add topics selected before inspecting outputs. Two domain readers should label paper relevance against explicit query goals, with disagreements retained. The same frozen candidate/retrieval conditions should be used for a simple OpenAlex baseline and each Nomad variant. Relevance should be separated from foundational value, novelty and project feasibility, rather than collapsed into one unreviewed label.

At a declared cutoff k, precision is

$$P@k = \frac{1}{k}\sum_{i=1}^{k} \mathbf{1}\{\text{paper } i \text{ is relevant}\}. \tag{6}$$

Graded judgments can support discounted cumulative gain,

$$DCG@k = \sum_{i=1}^{k} \frac{2^{r_i}-1}{\log_2(i+1)}, \qquad nDCG@k = \frac{DCG@k}{IDCG@k}, \tag{7}$$

provided the grading rubric and ideal ordering are fixed before comparison. These are proposed metrics; no values are supplied because the labels do not exist. Recall needs an adequately defined relevant universe and cannot be estimated merely by counting retrieved works.

An ablation should compare deterministic ranking, semantic augmentation, judge augmentation and shared-reference support on the same retained inputs. A separate small user experiment could measure time to select defensible starting papers, correction rate and explanation use, with task order counterbalanced. Graph usability needs its own small/typical/dense-map tests and accessibility review. These experiments would answer whether inspectability and extra structure help researchers; current component tests cannot substitute for them.

9 Conclusions

1. The pinned implementation converts OpenAlex candidates into traceable rankings and research-map views through the component boundaries in Fig. 1 and Table 1. This supports an implemented software contribution.

2. The retained test runner reported 88 passes in nine files. Three topic cases returned before their topic assertions, leaving 85 other reported case results; 85 is a subtraction of case counts, not an independently instrumented assertion count (Table 2).

3. Only two of five named topic captures exist. Their cluster-splitting and canonical-family checks are narrow regression invariants, not precision/recall or ranking-quality estimates (Table 3).

4. The documented arithmetic makes ranking inspectable, but the suite does not establish retrieval superiority, semantic-model benefit or researcher productivity. The engineering recommendation is to make missing-fixture coverage explicit before adding claims about output quality.

10 Future work

1. First make fixture activation testable at low cost: report missing captures as explicit skips or required-fixture failures, capture the three absent topics, and retain a fresh run with actual assertion/skip coverage.

2. Freeze a small, independently judged relevance set and compare deterministic ranking and optional augmentations against a simple OpenAlex baseline using the same captured inputs and the metrics defined above.

3. If that comparison is useful, run one counterbalanced researcher task study with paper-selection time, correction rate and map accessibility as distinct outcomes.

References

[1] M. Ulziibayar, *Nomad*, source, evaluation notes and fixture suite, commit be6bf7c836e1, 2026. `https://github.com/500ft/Nomad`.

[2] J. Priem, H. Piwowar and R. Orr, "OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts," 2022. `https://arxiv.org/abs/2205.01833`.

[3] OpenAlex, *Works overview and work attributes*, official data documentation, accessed 11 November 2026. `https://help.openalex.org/data/works/`; `https://help.openalex.org/data/works/attributes/`.